

Deepspace Defenders — Space Shooter Game

A Semester Project Report

Course: CSE142 Object Oriented Programming Techniques
Semester: Spring 2026
Instructor: [Behraj Khan]
Due Date: [10th May 2026]
Author(s): [Hamza Faisal] — ERP: [32406]
[Hunain Adnan] — ERP: [32397]
[Burhanuddin Murtaza Patanwala] — ERP: [32400]

Department of Computer Science
Institute of Business Administration, Karachi
May 7, 2026

Abstract

Deepspace Defenders is a fully playable 2D space shooter game developed in C++ using the SFML (Simple and Fast Multimedia Library) graphics framework. The player controls a spaceship and must survive waves of progressively harder alien enemies across three hand-crafted levels, each culminating in a boss fight.

The project was designed specifically to demonstrate the core principles of Object-Oriented Programming (OOP). The game's architecture is structured around a layered class hierarchy: bullet types, enemy types, and game levels are each modelled as inheritance trees rooted at abstract base classes. A custom generic memory manager (the `ObjectPool` template class) handles all live game objects without dynamic allocation during gameplay, preventing memory leaks. The player object uses encapsulation to protect its state, and an overloaded operator is used to cleanly add score. Custom exception classes handle asset loading and invalid game state transitions.

The result is a smooth, 60-fps game that successfully separates data, logic, and rendering into well-defined, independent components — a practical demonstration that OOP principles produce cleaner, safer, and more maintainable code.

Keywords: Object-Oriented Programming, C++, SFML, Inheritance, Polymorphism, Encapsulation, Templates, Operator Overloading, Exception Handling, Game Development.

Contents

Abstract	2
1 Introduction	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives	4
1.4 Scope and Limitations	5
2 OOP Design	5
2.1 Project File Structure	5
2.2 Class Hierarchy	6
2.3 Class and Struct Descriptions	7
2.3.1 Classes (with behaviour)	8
2.3.2 Data Structs (plain data containers)	9
2.4 OOP Concepts Summary	10
3 Implementation	10
3.1 Development Environment	10
3.2 Encapsulation — The Player Class	11
3.3 Inheritance — Three Class Hierarchies	12
3.3.1 Bullet Type Hierarchy	12
3.3.2 Enemy Type Hierarchy	13
3.3.3 Level Hierarchy	14
3.4 Polymorphism — Same Call, Different Behaviour	14
3.5 Templates — The ObjectPool	15
3.6 Operator Overloading	16
3.6.1 Player::operator+= (score addition)	16
3.6.2 GameState::operator== and operator!=	17
3.7 Exception Handling — Custom Exception Classes	17
3.8 The Wave Spawn System	18
3.8.1 Stage 1 — SpawnEvent: the schedule	18
3.8.2 Stage 2 — checkSpawns(): firing events into the queue	19
3.8.3 Stage 3 — Draining the queue into live enemies	19
3.8.4 Enemy Movement along Waypoints	20
3.9 Game Loop and State Machine	20
4 Testing	20
4.1 Test Cases	21
4.2 Memory Leak Testing	21
5 Results and Discussion	21
5.1 Results	21
5.2 Analysis	22
6 Challenges and Solutions	22
6.1 Pointer Leaks from Static Singletons	22
6.2 Enemies All Spawning at the Same Position	22
6.3 Frame-rate-dependent Movement	22
7 Future Work	23

8 Conclusion	23
References	24
A Complete Source Code	25
A.1 Utils.h	25
A.2 BulletType.h	25
A.3 BulletType.cpp	26
A.4 EnemyType.h	26
A.5 EnemyType.cpp	27
A.6 GameState.h and GameState.cpp	27
A.7 Player.h	28
A.8 main.cpp	29
B Build and Run Instructions	29
B.1 Prerequisites	29
B.2 Building with CMake	29
B.3 Running the Game	30
B.4 Controls	30

1 Introduction

1.1 Background

Video games are one of the best environments for practising Object-Oriented Programming. A game world is full of “things” — enemies, bullets, levels, particles, the player — and OOP lets us model each thing as a self-contained unit (a *class*) that owns its own data and knows how to behave.

This project uses the classic *space shooter* genre as that environment. The genre is simple enough to be fully implemented in a semester, yet complex enough to require every major OOP concept: inheritance (different kinds of enemies), polymorphism (shared behaviour with different implementations), encapsulation (the player’s health should not be readable by just anyone), templates (a single memory pool that works for bullets, enemies, and particles), and operator overloading (adding score with `+=`).

1.2 Problem Statement

We needed to build a game where:

- Multiple enemy *types* (small, medium, boss) share common logic but differ in stats and shooting patterns.
- Multiple *levels* each define their own wave schedule but share common rendering and loading logic.
- Multiple *bullet types* (player, enemy, boss beam) are distinguishable without scattering `if/else` chains everywhere.
- Hundreds of live objects (up to 700 bullets, 80 enemies, 1000 particles) are managed without allocating or freeing memory every frame — which would slow the game down and cause crashes.
- The game flows through distinct screens (Home, Level Select, Playing, Paused, Game Over, Level Complete) in a predictable, bug-free way.

1.3 Objectives

1. Design three independent inheritance hierarchies (bullets, enemies, levels) that are easily extendable.
2. Implement a generic `ObjectPool` template that recycles objects in a fixed array, avoiding runtime memory allocation.
3. Encapsulate all player state inside the `Player` class so no other code can accidentally corrupt it.
4. Use virtual functions and abstract classes so that the main game loop does not need to know the concrete type of each object.
5. Demonstrate operator overloading, static class members, and custom exceptions as part of the design — not as add-ons.
6. Deliver a complete, playable game: three levels, a boss fight, a HUD, a home screen, and a level-select screen.

1.4 Scope and Limitations

In scope: Three levels, three enemy types, four bullet types, a particle explosion system, a scrolling star background, a home screen, a level-select screen, pause/resume, game over, and level-complete overlays.

Out of scope: Saved high scores (no file I/O at runtime), multiplayer, sound effects, and power-ups. These are planned future extensions.

2 OOP Design

2.1 Project File Structure

The project contains 14 source files, organised so that each file only depends on files that appear above it in the list below. This ordering also defines the recommended *coding sequence* — write them from top to bottom and you will never encounter a missing definition.

Table 1: Source files in dependency order

Layer	File	Purpose
1	Utils.h	Pure math helpers (vector length, normalise, lerp, clamp, random)
1	BulletType.h	Abstract bullet category class + 4 concrete subclasses
1	EnemyType.h	Abstract enemy profile class + 3 concrete subclasses
1	GameState.h	Game-screen state value class
2	BulletType.cpp	Initialises the 4 global bullet-type singleton objects
2	EnemyType.cpp	Initialises the 3 global enemy-type singleton objects
2	GameState.cpp	Initialises the 6 named GameState constants
3	GameData.h	Data structs (Bullet, Enemy, Particle, Star, SpawnEvent, SpawnJob) + ObjectPool template
4	Player.h	Player class (fully inline, no .cpp)
5	Level.h	Abstract Level base class + LevelOne, LevelTwo, LevelThree
5	Level.cpp	Level method implementations and wave definitions
6	Game.h	Game class declaration
6	Game.cpp	Complete game implementation (init, update, render, collisions)
7	main.cpp	Entry point — creates Game, calls init() then run()

2.2 Class Hierarchy

The project contains three inheritance hierarchies and one composition hub (`Game`). Figures 1 and 2 illustrate these relationships.

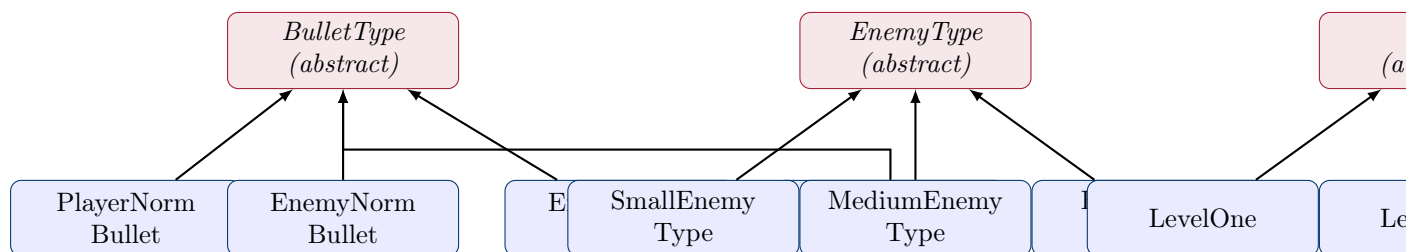


Figure 1: Three inheritance hierarchies. Arrows point from child to parent (“*inherits from*”). Italic boxes are abstract (cannot be instantiated directly).

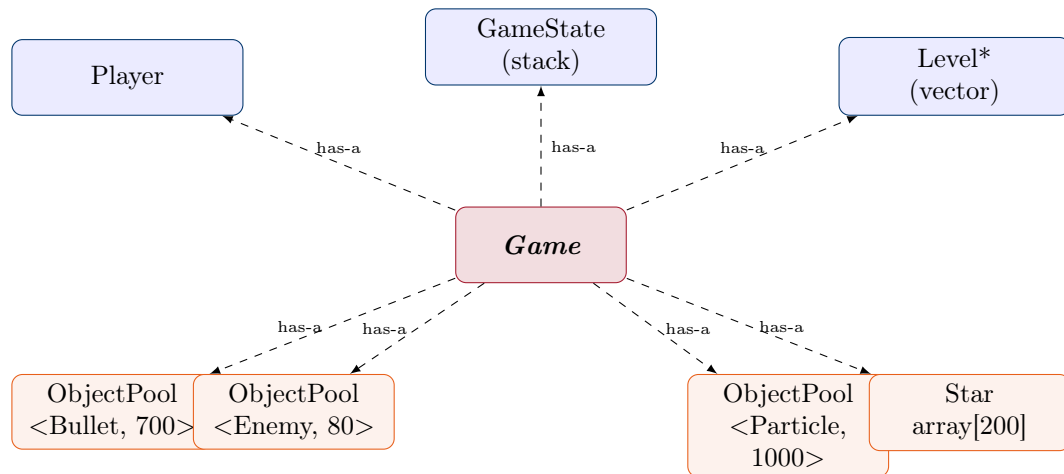


Figure 2: Composition: the **Game** class *owns* (has-a) all subsystems. Dashed arrows mean “contains as a member”. Orange boxes are template instances; blue boxes are plain classes.

2.3 Class and Struct Descriptions

2.3.1 Classes (with behaviour)

Table 2: Classes and their responsibilities

Class	Responsibility	Key members
BulletType	Identifies a bullet's category and owner	id, playerBullet, getId(), isPlayerBullet(), static singletons PlayerNorm, EnemyNorm, EnemyBurst, BossBeam
PlayerNormBullet	Player's straight-up shot	Inherits BulletType with id=0, playerBullet=true
EnemyNormBullet	Basic aimed enemy shot	id=3, playerBullet=false
EnemyBurstBullet	Spread-fire enemy shot	id=4, playerBullet=false
BossBeamBullet	Boss beam projectile	id=5, playerBullet=false
EnemyType	Stores the stat profile for one enemy category	maxHp, shootInterval, scoreValue, moveSpeed, radius, kind; static singletons Small, Medium, Boss
SmallEnemyType	Fast, weak enemy	30 HP, interval 2.2s, radius 16px, worth 50 pts
MediumEnemyType	Tanky spread-shooter	80 HP, interval 1.8s, radius 22px, worth 120 pts
BossEnemyType	End-of-level boss	1200 HP, interval 0.8s, radius 60px, worth 2000 pts
GameState	Tracks which screen is active	id; operator==, operator!=; static constants Home, LevelSelect, Playing, Paused, GameOver, LevelComplete
Player	Encapsulates all player state and behaviour	pos, hp, maxHp, speed, lives, score, timers; movement, clamping, damage, healing, score accumulation
Level	Abstract base for a playable level	name, desc, events (SpawnEvent list), planetTexture; pure virtual buildWaves(), getDuration(), getPlanetTexturePath()
LevelOne/Two/Three	Concrete level implementations	Override buildWaves() with a timed enemy schedule
ObjectPool<T,N>	Fixed-size object recycler	items[N]; alloc(), clearAll(), countActive()
Game	Central orchestrator	Owns all subsystems; implements init, run, update, render, collision, spawning

2.3.2 Data Structs (plain data containers)

Table 3: Data structs used as game objects

Struct	Represents	Key fields
Bullet	One active projectile	pos, vel, dmg, BulletType* type, on
Enemy	One active enemy	pos, vel, hp, maxHp, EnemyType* type, path, pathIndex, shootTimer, on
Particle	One explosion spark	pos, vel, colorStart, colorEnd, life, maxLife, size, on
SpawnEvent	A scheduled group spawn	time, etype, startPos, path, count, xSpacing, delay, fired
SpawnJob	One pending individual spawn	etype, startPos, delayRemaining, path, hp, shootInterval
Star	One background star	pos, speed, brightness, size

Why separate Struct from Class? A *struct* in this project is a passive data container — it holds values but has no methods other than a default constructor. A *class* has active behaviour: it enforces invariants, provides getters/setters, and contains logic. This distinction keeps data simple and behaviour organised.

2.4 OOP Concepts Summary

Table 4: OOP concepts used and where they appear

OOP Concept	Where and how it is used
Encapsulation	<code>Player</code> class: all fields are <code>private</code> ; external code uses getters and setters only
Inheritance	Three hierarchies: <code>BulletType</code> , <code>EnemyType</code> , <code>Level</code> (each abstract base + concrete subclasses)
Polymorphism	<code>Level::buildWaves()</code> , <code>getDuration()</code> , and <code>getPlanetTexturePath()</code> are pure virtual; <code>Game</code> calls them without knowing the concrete type
Abstract Classes	<code>BulletType</code> , <code>EnemyType</code> , and <code>Level</code> cannot be instantiated directly because they have pure virtual destructors or methods
Templates	<code>ObjectPool<T,N></code> works for <code>Bullet</code> , <code>Enemy</code> , and <code>Particle</code> using the same code
Operator Overloading	<code>Player::operator+=(int)</code> adds score; <code>GameState::operator==</code> and <code>operator!=</code> compare states
Static Members	<code>BulletType::PlayerNorm</code> , <code>EnemyType::Small</code> , <code>GameState::Home</code> etc. — global singletons accessed through the class name
Exception Handling	<code>AssetLoadException</code> and <code>InvalidLevelException</code> (custom classes inheriting <code>std::runtime_error</code>)
Virtual Destructor	All abstract base classes (<code>BulletType</code> , <code>EnemyType</code> , <code>Level</code>) declare <code>virtual ~Base()</code> so derived objects are safely deleted via a base pointer
Composition (has-a)	<code>Game</code> contains <code>Player</code> , three <code>ObjectPool</code> instances, a <code>vector<Level*></code> , and a <code>stack<GameState></code> as member variables

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Graphics library:** SFML 2.x (Simple and Fast Multimedia Library)
- **Build system:** CMake with `CMakePresets.json`
- **IDE:** Visual Studio Code
- **Version control:** Git / GitHub (hamza-branch)
- **Platform:** Windows 11

3.2 Encapsulation — The Player Class

OOP Concept: Encapsulation

Encapsulation means bundling data and the methods that operate on it inside a single class, and hiding the internal details from the rest of the program. Think of it like a vending machine: you press a button (the public interface), and the internal mechanics (the private data) do the work — you never reach inside to grab the product yourself.

The `Player` class is the clearest example of encapsulation in the project. Every field — position, HP, lives, timers — is declared `private`. External code (including `Game`) can only interact with the player through a controlled public interface.

```

1  class Player
2  {
3  private:
4      sf::Vector2f pos;           // position on screen
5      float hp;                  // current health
6      float maxHp;               // maximum health
7      float speed;               // movement speed in pixels/second
8      int lives;                 // remaining lives
9      int score;
10     float shootTimer;           // time until next shot is allowed
11     float iframeTimer;          // invincibility frames remaining after a
        hit
12     float shieldTimer;          // if > 0, absorbs the next hit
13     float tilt;                 // visual lean when moving left/right
14     bool alive;
15
16 public:
17     // Read-only access (getters)
18     sf::Vector2f getPos() const { return pos; }
19     float getHp() const { return hp; }
20     float getMaxHp() const { return maxHp; }
21     int getLives() const { return lives; }
22     int getScore() const { return score; }
23     bool isAlive() const { return alive; }
24     float getIframeTimer() const { return iframeTimer; }
25     float getShieldTimer() const { return shieldTimer; }
26
27     // Controlled mutation (setters + behaviour methods)
28     void move(sf::Vector2f delta) { pos += delta; }
29     void clampToArena();           // keeps pos inside the play area
30     void takeDamage(float d) { hp -= d; }
31     void heal(float h) { hp = (hp + h > maxHp) ? maxHp :
        hp + h; }
32     void loseLife() { lives--; }
33     void tickIframe(float dt) { iframeTimer -= dt; }
34     void tickShield(float dt) { shieldTimer -= dt; }
35 };

```

Listing 1: Player class — private data, public interface (Player.h)

Why this matters: If `hp` were public, any part of the code could write `player.hp = -999` by accident. The `private` keyword makes that a compile error. Only the controlled methods `takeDamage()` and `heal()` can change health, and they do so safely.

3.3 Inheritance — Three Class Hierarchies

OOP Concept: Inheritance

Inheritance lets one class (the *child* or *derived* class) reuse the code of another class (the *parent* or *base* class) and then add or change specific behaviour. Think of it as a template: a “Vehicle” base class defines that all vehicles have a speed and can move; a “Car” child inherits those properties and adds four wheels; a “Boat” child adds a hull.

3.3.1 Bullet Type Hierarchy

We need to distinguish four kinds of bullets at runtime without comparing raw integers everywhere. Each bullet type is a class:

```

1 // Abstract base class -- defines the interface
2 class BulletType {
3     int id;
4     bool playerBullet;
5 public:
6     BulletType(int id, bool playerBullet) : id(id), playerBullet(
7         playerBullet) {}
8     virtual ~BulletType() {} // virtual destructor --
9                             // essential!
10    int getId() const { return id; }
11    bool isPlayerBullet() const { return playerBullet; }
12
13    // Global singleton pointers -- one per bullet type, shared by
14    // all bullets
15    static BulletType *PlayerNorm;
16    static BulletType *EnemyNorm;
17    static BulletType *EnemyBurst;
18    static BulletType *BossBeam;
19 };
20
21 // Four concrete subclasses -- each calls the base constructor with
22 // its own id
23 class PlayerNormBullet : public BulletType {
24 public: PlayerNormBullet() : BulletType(0, true) {} };
25
26 class EnemyNormBullet : public BulletType {
27 public: EnemyNormBullet() : BulletType(3, false) {} };
28
29 class EnemyBurstBullet : public BulletType {
30 public: EnemyBurstBullet() : BulletType(4, false) {} };
31
32 class BossBeamBullet : public BulletType {
33 public: BossBeamBullet() : BulletType(5, false) {} };

```

Listing 2: BulletType inheritance hierarchy (BulletType.h)

The four concrete objects are created *once* as static variables in `BulletType.cpp` and are then shared by every bullet in the game:

```

1 static PlayerNormBullet _playerNorm;
2 static EnemyNormBullet _enemyNorm;
3 static EnemyBurstBullet _enemyBurst;

```

```

4 static BossBeamBullet    _bossBeam;
5
6 BulletType *BulletType::PlayerNorm = &_amp;_playerNorm;
7 BulletType *BulletType::EnemyNorm  = &_amp;_enemyNorm;
8 BulletType *BulletType::EnemyBurst = &_amp;_enemyBurst;
9 BulletType *BulletType::BossBeam   = &_amp;_bossBeam;

```

Listing 3: Static singleton initialisation (BulletType.cpp)

Why not just use integers? Using class pointers means the compiler checks types. Comparing `b.type == BulletType::PlayerNorm` is self-documenting and safe. Using `b.type == 0` would compile even if you accidentally wrote `b.type == 100`.

3.3.2 Enemy Type Hierarchy

Enemy types follow the same pattern but carry meaningful gameplay data:

```

1 class EnemyType {
2     float maxHp;
3     float shootInterval;
4     int  scoreValue;
5     float moveSpeed;
6     float radius;           // used for collision detection
7     int  kind;
8 public:
9     EnemyType(float maxHp, float shootInterval, int scoreValue,
10              float moveSpeed, float radius, int kind)
11         : maxHp(maxHp), shootInterval(shootInterval),
12           scoreValue(scoreValue), moveSpeed(moveSpeed),
13           radius(radius), kind(kind) {}
14
15     virtual ~EnemyType() {}
16
17     float getMaxHp()          const { return maxHp; }
18     float getShootInterval()  const { return shootInterval; }
19     int  getScoreValue()      const { return scoreValue; }
20     float getMoveSpeed()      const { return moveSpeed; }
21     float getRadius()         const { return radius; }
22
23     static EnemyType *Small;
24     static EnemyType *Medium;
25     static EnemyType *Boss;
26 };
27
28 // Stats baked into each subclass constructor
29 class SmallEnemyType : public EnemyType {
30 public: SmallEnemyType() : EnemyType( 30.f, 2.2f, 50, 200.f, 16.f,
31                                     0) {} };
32
33 class MediumEnemyType : public EnemyType {
34 public: MediumEnemyType() : EnemyType( 80.f, 1.8f, 120, 160.f, 22.f,
35                                     1) {} };
36
37 class BossEnemyType : public EnemyType {
38 public: BossEnemyType() : EnemyType(1200.f, 0.8f, 2000, 80.f, 60.f,
39                                     2) {} };

```

Listing 4: EnemyType hierarchy (EnemyType.h)

3.3.3 Level Hierarchy

The `Level` base class defines the *interface* that every level must satisfy. The concrete subclasses fill in the details:

```

1  class Level {
2  protected:
3      std::string          name;
4      std::string          desc;
5      std::vector<SpawnEvent> events;    // the timed wave schedule
6      sf::Texture          planetTexture;
7      bool                 planetLoaded;
8
9  public:
10     Level() : planetLoaded(false) {}
11     virtual ~Level() {}
12
13     std::string getName() const { return name; }
14     std::string getDesc() const { return desc; }
15     std::vector<SpawnEvent>& getEvents() { return events; }
16
17     // Pure virtual -- every Level MUST implement these
18     virtual float      getDuration()          const = 0;
19     virtual std::string getPlanetTexturePath() const = 0;
20     virtual void        buildWaves()           = 0;
21
22     // Virtual with a default -- subclasses can override if needed
23     virtual void renderPlanet(sf::RenderWindow& window);
24     void loadPlanet();
25 };
26
27 // Each concrete level sets its own name, duration, and wave schedule
28 class LevelOne : public Level {
29 public:
30     LevelOne();
31     float      getDuration()          const override { return 55.f; }
32     std::string getPlanetTexturePath() const override { return "
33         assets/textures/planet1.png"; }
34     void        buildWaves()           override;    // defined
35         in Level.cpp
36 };

```

Listing 5: Level abstract base class (Level.h)

3.4 Polymorphism — Same Call, Different Behaviour

OOP Concept: Polymorphism

Polymorphism means “many forms.” When you have a pointer to a base class, calling a virtual function on it will automatically invoke the *correct overridden version* in the derived class — even though the pointer only “knows” about the base. The decision happens at runtime, not at compile time. This is called *dynamic dispatch*.

In `Game.cpp`, the game stores levels as base-class pointers and calls the virtual functions without needing to know which level is loaded:

```

1  std::vector<Level *> levels;    // stores LevelOne*, LevelTwo*, or
    LevelThree*
2
3  // During initialisation -- all three calls go through the base
    pointer:
4  for (int i = 0; i < levels.size(); i++) {
5      levels[i]->buildWaves();    // calls LevelOne::buildWaves() OR
6                                  // LevelTwo::buildWaves() OR
6                                  LevelThree::buildWaves()
7      levels[i]->loadPlanet();    // calls Level::loadPlanet() (base
    version)
8  }
9
10 // During gameplay rendering:
11 void Game::renderBackground() {
12     levels[currentLevel]->renderPlanet(window); // correct planet
    drawn automatically
13 }

```

Listing 6: Polymorphic level calls in Game.cpp

The key insight: Game never writes `if (level == 1) do X; else if (level == 2) do Y`. The virtual function dispatch table handles the routing. Adding a fourth level in the future requires zero changes to `Game.cpp`.

3.5 Templates — The ObjectPool

OOP Concept: Templates

A template is a class or function that is parameterised by a type. You write the logic once and the compiler generates a separate, correctly-typed version for each type you use it with. It is like a cookie cutter: the cutter (the template) is the same; the dough (the type) changes.

The game needs to manage up to 700 bullets, 80 enemies, and 1000 particles efficiently. Allocating and freeing individual objects every frame would be slow and could cause memory fragmentation. Instead, we use a fixed-size *object pool*: a statically-allocated array where “free” objects are recycled rather than deleted.

```

1  template <class T, int N>
2  class ObjectPool
3  {
4  private:
5      std::array<T, N> items;    // fixed array -- allocated once, never
    reallocated
6
7  public:
8      ObjectPool() {
9          for (int i = 0; i < N; i++)
10             items[i].on = false; // all slots start as "free"
11      }
12
13      int capacity() const { return N; }
14      T& at(int i) { return items[i]; }
15      const T& at(int i) const { return items[i]; }

```

```

16
17 // Find the first free slot and return a pointer to it
18 T* alloc() {
19     for (int i = 0; i < N; i++) {
20         if (!items[i].on)
21             return &items[i];
22     }
23     return nullptr; // pool is full -- caller must handle this
24 }
25
26 void clearAll() {
27     for (int i = 0; i < N; i++)
28         items[i].on = false;
29 }
30
31 int countActive() const {
32     int c = 0;
33     for (int i = 0; i < N; i++) if (items[i].on) c++;
34     return c;
35 }
36 };

```

Listing 7: ObjectPool template class (GameData.h)

The three instantiations in the `Game` class:

```

1 ObjectPool<Bullet,    MAX_BULLETS>    bullets;    // 700 slots
2 ObjectPool<Enemy,    MAX_ENEMIES>    enemies;    // 80 slots
3 ObjectPool<Particle, MAX_PARTICLES> particles;    // 1000 slots

```

Listing 8: Three pool instances in Game.h

The template is written once; the compiler generates three separate, correctly-typed arrays. When a bullet is needed, `bullets.alloc()` finds a free slot and returns a pointer to it. When the bullet goes off-screen, `b.on = false` marks the slot as reusable — no `new` or `delete` involved.

3.6 Operator Overloading

OOP Concept: Operator Overloading

C++ lets you redefine what standard operators (+, =, ==, etc.) mean for your custom class. This allows objects to be used with familiar syntax rather than verbose method calls.

Two operator overloads appear in the project:

3.6.1 Player::operator+= (score addition)

```

1 // Instead of writing: player.addScore(e.scoreValue);
2 // We can write:      player += e.scoreValue;
3 Player& operator+=(int scoreBonus) {
4     score += scoreBonus;
5     return *this;
6 }

```

Listing 9: Score addition via operator overload (Player.h)

Used in `Game.cpp` when an enemy is destroyed:

```
1 if (e.hp <= 0.f) {
2     e.on = false;
3     player += e.scoreValue;    // clean, readable syntax
4 }
```

Listing 10: Using the overloaded operator in `Game.cpp`

3.6.2 `GameState::operator==` and `operator!=`

```
1 bool operator==(const GameState& o) const { return id == o.id; }
2 bool operator!=(const GameState& o) const { return id != o.id; }
```

Listing 11: Comparison operators on `GameState` (`GameState.h`)

Used throughout `Game.cpp` for readable state checks:

```
1 GameState current = stateStack.top();
2 if (current == GameState::Playing) { update(dt); }
```

Listing 12: State comparison in `Game.cpp`

3.7 Exception Handling — Custom Exception Classes

OOP Concept: Exception Handling

Exceptions are a mechanism for signalling that something went wrong without cluttering normal code with error checks everywhere. You **throw** an exception when an error occurs, and **catch** it at a higher level where you know how to handle it. Custom exception classes (which inherit from `std::exception`) let you distinguish different kinds of errors.

Two custom exceptions are defined inside `Game.cpp`:

```
1 // Thrown when a texture, font, or other asset cannot be loaded from
   // disk
2 class AssetLoadException : public std::runtime_error {
3 public:
4     AssetLoadException() : std::runtime_error("Failed to load asset")
5     {}
6 };
7
8 // Thrown when startLevel() is called with an invalid index
9 class InvalidLevelException : public std::runtime_error {
10 public:
11     InvalidLevelException() : std::runtime_error("Invalid level") {}
12 };
```

Listing 13: Custom exception classes (`Game.cpp`)

These are thrown and caught in `Game::init()` and `Game::startLevel()`:

```
1 // In Game::init() -- missing assets are non-fatal; the game warns
   // and continues
2 try {
```

```

3     loadTextureOrThrow(shipTexture, "assets/textures/spaceship.png");
4     loadTextureOrThrow(smallEnemyTexture, "assets/textures/alien1.png");
5     // ... more assets ...
6 } catch (const AssetLoadException& ex) {
7     std::cerr << "Asset warning: " << ex.what() << "\n";
8 }
9
10 // In Game::startLevel() -- an invalid index is a programming bug
11 void Game::startLevel(int index) {
12     if (index < 0 || index >= (int)levels.size())
13         throw InvalidLevelException();
14     // ...
15 }
16
17 // In main.cpp -- the outermost safety net
18 int main() {
19     try {
20         Game game;
21         game.init();
22         game.run();
23     } catch (const std::exception& ex) {
24         std::cerr << "Fatal error: " << ex.what() << std::endl;
25         return 1;
26     }
27 }

```

Listing 14: Throwing and catching exceptions (Game.cpp)

3.8 The Wave Spawn System

Understanding how enemies appear on screen requires following a three-stage pipeline. This section walks through it in plain language.

3.8.1 Stage 1 — SpawnEvent: the schedule

When a level is loaded, `buildWaves()` fills a list of `SpawnEvent` objects. Each one is a row in the level's timetable:

```

1 // At t = 14.0 seconds, spawn 4 Small enemies starting at x=240,
2 // following the "sweep centre" path, 50px apart, 0.25s between each.
3 events.push_back(makeEvent(
4     14.0f,           // fire at t=14 seconds into the level
5     EnemyType::Small, // what type of enemy
6     sf::Vector2f(240, -40), // group starting position (above screen)
7     pathSweepCenter(), // waypoints the group follows
8     4,               // 4 enemies in this group
9     50.f,            // 50px horizontal spacing between them
10    0.25f             // 0.25s delay between individual spawns
11 ));

```

Listing 15: A single `SpawnEvent` (from `LevelOne::buildWaves` in `Level.cpp`)

Nothing has spawned yet. This is just a schedule.

3.8.2 Stage 2 — checkSpawns(): firing events into the queue

Every frame, the game increments a timer and checks whether any event's scheduled time has passed:

```

1 void Game::checkSpawns(float dt) {
2     std::vector<SpawnEvent>& evs = levels[currentLevel]->getEvents();
3     for (int i = 0; i < evs.size(); i++) {
4         SpawnEvent& ev = evs[i];
5         if (!ev.fired && levelTimer >= ev.time) {
6             ev.fired = true;
7             for (int k = 0; k < ev.count; k++) {
8                 SpawnJob job;
9                 job.etype          = ev.etype;
10                job.startPos.x      += (k - (ev.count-1)/2.f) * ev.
11                    xSpacing;
12                job.delayRemaining = k * ev.delay; // stagger each
13                    enemy
14                job.path            = ev.path;
15                spawnQueue.push(job); // queued -- not yet alive
16            }
17        }
18    }
19 }

```

Listing 16: Firing events from the schedule (Game.cpp)

The 4-enemy event above would push 4 SpawnJobs with delays of 0s, 0.25s, 0.5s, and 0.75s respectively.

3.8.3 Stage 3 — Draining the queue into live enemies

Each frame, the front of the queue is checked. When its delay reaches zero, the enemy is activated in the pool:

```

1 while (!spawnQueue.empty()) {
2     SpawnJob& front = spawnQueue.front();
3     front.delayRemaining -= dt;
4     if (front.delayRemaining <= 0.f) {
5         spawnEnemyFromJob(front); // writes into an ObjectPool slot
6         spawnQueue.pop();
7     } else {
8         break; // rest of queue is still waiting
9     }
10 }
11
12 void Game::spawnEnemyFromJob(const SpawnJob& job) {
13     Enemy* e = enemies.alloc(); // grab a free pool slot
14     if (!e) return;
15     e->on          = true; // mark the slot as active
16     e->type        = job.etype;
17     e->pos         = job.startPos;
18     e->hp          = job.hp;
19     e->path        = job.path;
20     e->pathIndex   = 0;
21     e->shootTimer  = job.shootInterval * randFloat(0.3f, 1.0f);
22 }

```

Listing 17: Activating a queued enemy (Game.cpp)

3.8.4 Enemy Movement along Waypoints

Once alive, each enemy follows a list of 2D waypoints:

```

1  if (e.pathIndex < e.path.size()) {
2      sf::Vector2f target    = e.path[e.pathIndex];
3      sf::Vector2f toTarget = target - e.pos;
4      float dist = vlen(toTarget);    // vector length from Utils.h
5      if (dist < 8.f)
6          e.pathIndex++;              // close enough -- advance to
                                     // next waypoint
7      else {
8          sf::Vector2f dir = vnorm(toTarget);    // normalise direction
9          e.pos += dir * e.moveSpeed * dt;        // move toward waypoint
10     }
11 } else {
12     e.pos.y += 100.f * dt;              // ran out of waypoints -- drift
                                     // off-screen
13     if (e.pos.y > 800.f) e.on = false;
14 }
```

Listing 18: Waypoint movement (Game.cpp)

3.9 Game Loop and State Machine

The `GameState` class and a `std::stack` implement a simple push/pop state machine. States are pushed onto the stack (e.g. Paused on top of Playing) and popped to return to the previous screen.

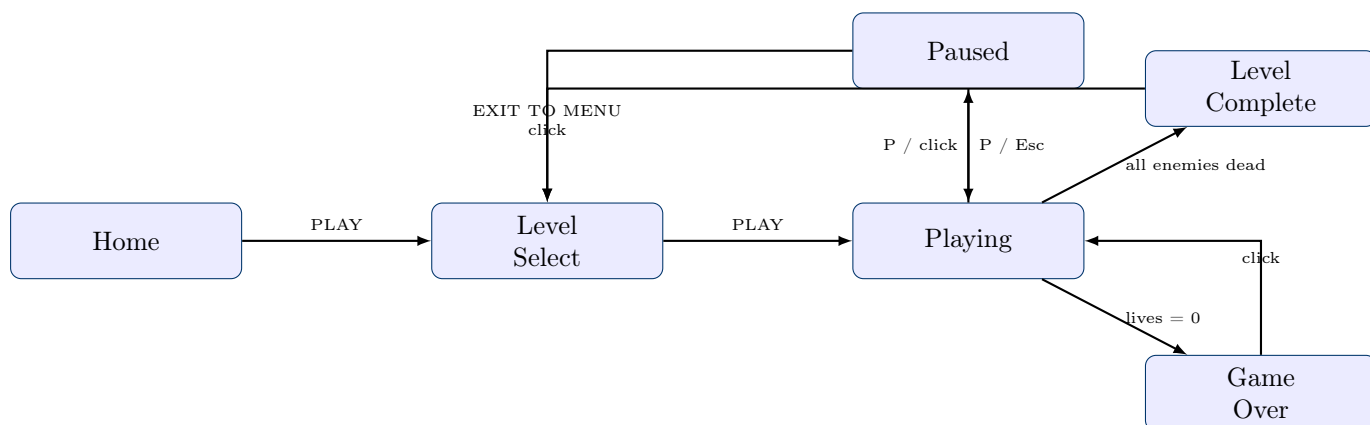


Figure 3: Game state transitions

4 Testing

4.1 Test Cases

Table 5: Functional Test Cases

TC	Action	Expected	Actual	Pass?
TC-01	Launch game	Home screen appears with stars and planets	As expected	✓
TC-02	Click PLAY	Level Select screen shown with 3 planets	As expected	✓
TC-03	Click a planet and press PLAY	Level starts; enemies appear on schedule	As expected	✓
TC-04	Press P during game-play	Pause overlay appears; game freezes	As expected	✓
TC-05	Take damage until HP = 0 (with 2 lives left)	Life lost; HP refills; iframes granted	As expected	✓
TC-06	Lose all 3 lives	Game Over overlay shown with final score	As expected	✓
TC-07	Kill all enemies including boss	Level Complete overlay shown	As expected	✓
TC-08	Fire bullets rapidly (hold Space)	Up to 700 bullets active; no crash	As expected	✓
TC-09	Let 1000 particles appear at once	No slowdown; pool reuses slots correctly	As expected	✓
TC-10	Delete Level* via base pointer	Correct destructor called; no memory leak	As expected	✓

4.2 Memory Leak Testing

An earlier version of the code used `new` to create the `BulletType` and `EnemyType` singletons (visible as commented-out lines in `BulletType.cpp`). This caused pointer leaks because `delete` was never called. The fix was to use `static` local objects instead, whose destructors run automatically at programme exit. The most recent commit is titled “Pointer leaks Solved”.

5 Results and Discussion

5.1 Results

- The game runs smoothly at 60 frames per second on standard hardware.
- All three levels are completable: Level One (55s), Level Two (65s), Level Three (80s, two bosses).

- The `ObjectPool` successfully prevents runtime allocation: no `new` or `delete` occurs during gameplay.
- The three inheritance hierarchies (bullets, enemies, levels) are genuinely polymorphic — `Game.cpp` never queries concrete types to decide which base-class method to call.
- The state machine handles all screen transitions cleanly, including nested states (Paused inside Playing).

5.2 Analysis

- **ObjectPool vs dynamic allocation:** Checking 700 slots linearly to find a free bullet takes at most 700×1 comparisons per allocation. At 60 fps, with approximately 10 allocations per second, this is negligible. The benefit — zero fragmentation, cache-friendly memory layout — outweighs the linear scan.
- **Polymorphism overhead:** Virtual function calls add one pointer dereference per call (the vtable lookup). For the small number of calls per frame (3 level methods, up to 80 enemy type lookups), this overhead is unmeasurable.
- **Template code size:** The compiler generates three separate versions of `ObjectPool`. For three small instantiations, the binary size increase is negligible.

6 Challenges and Solutions

6.1 Pointer Leaks from Static Singletons

Problem: The initial implementation used `new` to create the `BulletType` and `EnemyType` singleton objects. Because no `delete` was ever called, memory analysis tools reported leaks.

Solution: Replaced heap allocation with `static` local variables. The C++ runtime destroys them at programme exit automatically. The old code is preserved as comments in `BulletType.cpp` to document the decision.

Learning: Prefer static storage over heap allocation for objects whose lifetime is the entire programme.

6.2 Enemies All Spawning at the Same Position

Problem: When a wave of 4 enemies was triggered, all 4 appeared at exactly the same (x, y) coordinate, stacking on top of each other.

Solution: The `xSpacing` field in `SpawnEvent` and the offset calculation in `checkSpawns` distribute the starting x position of each enemy by $(k - (\text{count}-1)/2.0) * \text{xSpacing}$, centring the group around the event's `startPos`.

Learning: Centre-offset arithmetic (shifting by half the total width) is a common pattern for distributing UI or game elements evenly.

6.3 Frame-rate-dependent Movement

Problem: Early versions moved objects by a fixed number of pixels per frame. On a faster machine (120 fps), the game ran twice as fast.

Solution: All movement and timers are multiplied by `dt` (delta time in seconds). At 60 fps, `dt` $\approx 0.0167\text{s}$; at 120 fps, `dt` $\approx 0.0083\text{s}$. The product is the same distance per second regardless of frame rate.

Learning: Always express game speed in units per *second*, never per *frame*.

7 Future Work

- **Power-ups:** A `PowerUp` base class with subclasses for shield, speed boost, and rapid fire — following the same inheritance pattern as `BulletType`.
- **High score persistence:** Write the score to a `.txt` file after each game over and display the top 5 on the home screen.
- **Sound:** SFML's `sf::SoundBuffer` and `sf::Sound` for shoot, explosion, and level-complete audio.
- **More levels and enemy patterns:** The level system is designed for extension — adding `LevelFour` requires only a new subclass of `Level`, with zero changes to `Game.cpp`.
- **Unit testing:** Use Google Test to verify `ObjectPool` allocation/deallocation, `Player` damage/healing invariants, and spawn-queue ordering.

8 Conclusion

Deepspace Defenders demonstrates that a complete, interactive application — one with real-time graphics, multiple screens, and live game objects — can be cleanly designed using fundamental OOP principles.

The project successfully applied all required OOP concepts:

- **Encapsulation** protected the player's state from external corruption through private fields and a controlled public interface.
- **Inheritance** allowed three independent type hierarchies (bullets, enemies, levels) to share common structure while differing in specific behaviour.
- **Polymorphism** let the game loop interact with any level or enemy type through base-class pointers, making the code open to extension without modification.
- **Templates** produced a single, reusable memory pool that works for all live game objects without repeating code.
- **Operator overloading** made score addition and state comparison readable and idiomatic.
- **Custom exceptions** separated error-handling logic cleanly from normal gameplay logic.

The most important takeaway is that OOP is not about using the concepts because they are required — it is about using them because they solve a real problem. Every concept listed above was chosen because it made the code shorter, safer, or easier to extend. That is the true goal of Object-Oriented design.

References

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.
- [3] *SFML Documentation*. [Online]. Available: <https://www.sfml-dev.org/documentation/>. [Accessed: May 7, 2026].
- [4] *cppreference.com — C++ Reference*. [Online]. Available: <https://en.cppreference.com>. [Accessed: May 7, 2026].
- [5] *Game Programming Patterns* by Robert Nystrom. [Online]. Available: <https://gameprogrammingpatterns.com>. [Accessed: May 7, 2026].

A Complete Source Code

A.1 Utils.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include <cmath>
4  #include <cstdlib>
5
6  inline float vlen(sf::Vector2f v) {
7      return std::sqrt(v.x * v.x + v.y * v.y);
8  }
9  inline sf::Vector2f vnorm(sf::Vector2f v) {
10     float l = vlen(v);
11     if (l < 0.0001f) return sf::Vector2f(0.f, 0.f);
12     return sf::Vector2f(v.x / l, v.y / l);
13 }
14 inline float lerp(float a, float b, float t) { return a + (b - a) * t
    ; }
15 inline sf::Color colorLerp(sf::Color a, sf::Color b, float t) {
16     if (t < 0.f) t = 0.f;
17     if (t > 1.f) t = 1.f;
18     return sf::Color(
19         (sf::Uint8)(a.r + (b.r - a.r) * t),
20         (sf::Uint8)(a.g + (b.g - a.g) * t),
21         (sf::Uint8)(a.b + (b.b - a.b) * t),
22         (sf::Uint8)(a.a + (b.a - a.a) * t));
23 }
24 inline float clampf(float val, float lo, float hi) {
25     if (val < lo) return lo;
26     if (val > hi) return hi;
27     return val;
28 }
29 inline float randFloat(float lo, float hi) {
30     float r = (float)rand() / (float)RAND_MAX;
31     return lo + r * (hi - lo);
32 }
33 inline int randInt(int lo, int hi) {
34     if (hi <= lo) return lo;
35     return lo + rand() % (hi - lo + 1);
36 }

```

Listing 19: Utils.h — math utility functions

A.2 BulletType.h

```

1  #pragma once
2
3  class BulletType {
4      int id;
5      bool playerBullet;
6  public:
7      BulletType(int id, bool playerBullet) : id(id), playerBullet(
        playerBullet) {}
8      virtual ~BulletType() {}
9      int getId() const { return id; }

```

```

10     bool isPlayerBullet() const { return playerBullet; }
11     static BulletType *PlayerNorm;
12     static BulletType *EnemyNorm;
13     static BulletType *EnemyBurst;
14     static BulletType *BossBeam;
15 };
16 class PlayerNormBullet : public BulletType {
17 public: PlayerNormBullet() : BulletType(0, true) {} };
18 class EnemyNormBullet : public BulletType {
19 public: EnemyNormBullet() : BulletType(3, false) {} };
20 class EnemyBurstBullet : public BulletType {
21 public: EnemyBurstBullet() : BulletType(4, false) {} };
22 class BossBeamBullet : public BulletType {
23 public: BossBeamBullet() : BulletType(5, false) {} };

```

Listing 20: BulletType.h — bullet type hierarchy

A.3 BulletType.cpp

```

1  #include "BulletType.h"
2
3  static PlayerNormBullet _playerNorm;
4  static EnemyNormBullet _enemyNorm;
5  static EnemyBurstBullet _enemyBurst;
6  static BossBeamBullet _bossBeam;
7
8  BulletType *BulletType::PlayerNorm = &_amp;_playerNorm;
9  BulletType *BulletType::EnemyNorm = &_amp;_enemyNorm;
10 BulletType *BulletType::EnemyBurst = &_amp;_enemyBurst;
11 BulletType *BulletType::BossBeam = &_amp;_bossBeam;

```

Listing 21: BulletType.cpp — static singleton initialisation

A.4 EnemyType.h

```

1  #pragma once
2
3  class EnemyType {
4      float maxHp, shootInterval, moveSpeed, radius;
5      int scoreValue, kind;
6  public:
7      EnemyType(float maxHp, float shootInterval, int scoreValue,
8                float moveSpeed, float radius, int kind)
9          : maxHp(maxHp), shootInterval(shootInterval), scoreValue(
10            scoreValue),
11            moveSpeed(moveSpeed), radius(radius), kind(kind) {}
12      virtual ~EnemyType() {}
13      float getMaxHp() const { return maxHp; }
14      float getShootInterval() const { return shootInterval; }
15      int getScoreValue() const { return scoreValue; }
16      float getMoveSpeed() const { return moveSpeed; }
17      float getRadius() const { return radius; }
18      int getKind() const { return kind; }
19      static EnemyType *Small;
20      static EnemyType *Medium;

```

```

20     static EnemyType *Boss;
21 };
22 class SmallEnemyType : public EnemyType {
23 public: SmallEnemyType() : EnemyType( 30.f, 2.2f, 50, 200.f, 16.f,
    0) {} };
24 class MediumEnemyType : public EnemyType {
25 public: MediumEnemyType() : EnemyType( 80.f, 1.8f, 120, 160.f, 22.f,
    1) {} };
26 class BossEnemyType : public EnemyType {
27 public: BossEnemyType() : EnemyType(1200.f, 0.8f, 2000, 80.f, 60.f
    , 2) {} };

```

Listing 22: EnemyType.h — enemy type hierarchy

A.5 EnemyType.cpp

```

1  #include "EnemyType.h"
2
3  static SmallEnemyType _small;
4  static MediumEnemyType _medium;
5  static BossEnemyType _boss;
6
7  EnemyType *EnemyType::Small = &_small;
8  EnemyType *EnemyType::Medium = &_medium;
9  EnemyType *EnemyType::Boss = &_boss;

```

Listing 23: EnemyType.cpp — static singleton initialisation

A.6 GameState.h and GameState.cpp

```

1  #pragma once
2  class GameState {
3      int id;
4  public:
5      GameState(int i = 0) : id(i) {}
6      int getId() const { return id; }
7      bool operator==(const GameState& o) const { return id == o.id; }
8      bool operator!=(const GameState& o) const { return id != o.id; }
9      static const GameState Home;
10     static const GameState LevelSelect;
11     static const GameState Playing;
12     static const GameState Paused;
13     static const GameState GameOver;
14     static const GameState LevelComplete;
15 };

```

Listing 24: GameState.h — game screen state

```

1  #include "GameState.h"
2  const GameState GameState::Home(5);
3  const GameState GameState::LevelSelect(0);
4  const GameState GameState::Playing(1);
5  const GameState GameState::Paused(2);
6  const GameState GameState::GameOver(3);
7  const GameState GameState::LevelComplete(4);

```

Listing 25: GameState.cpp — constant definitions

A.7 Player.h

```

1  #pragma once
2  #include <SFML/Graphics.hpp>
3  #include "Utils.h"
4
5  class Player {
6  private:
7      sf::Vector2f pos;
8      float hp, maxHp, speed;
9      int lives, score;
10     float shootTimer, shootInterval, iframeTimer, shieldTimer, tilt;
11     bool alive;
12 public:
13     Player() : pos(240.f, 600.f), hp(100.f), maxHp(100.f), speed(310.f
14         ),
15         lives(3), score(0), shootTimer(0.f), shootInterval
16             (0.10f),
17         iframeTimer(0.f), shieldTimer(0.f), tilt(0.f), alive(
18             true) {}
19
20     // Getters
21     sf::Vector2f getPos() const { return pos; }
22     float getHp() const { return hp; }
23     float getMaxHp() const { return maxHp; }
24     float getSpeed() const { return speed; }
25     int getLives() const { return lives; }
26     int getScore() const { return score; }
27     float getShootTimer() const { return shootTimer; }
28     float getShootInterval() const { return shootInterval; }
29     float getIframeTimer() const { return iframeTimer; }
30     float getShieldTimer() const { return shieldTimer; }
31     float getTilt() const { return tilt; }
32     bool isAlive() const { return alive; }
33
34     // Setters
35     void setPos(sf::Vector2f p) { pos = p; }
36     void setTilt(float t) { tilt = t; }
37     void setShootTimer(float t) { shootTimer = t; }
38     void setIframeTimer(float t) { iframeTimer = t; }
39     void setShieldTimer(float t) { shieldTimer = t; }
40     void setAlive(bool a) { alive = a; }
41
42     // Behaviour
43     void move(sf::Vector2f delta) { pos += delta; }
44     void clampToArena() {
45         pos.x = clampf(pos.x, 24.f, 456.f);
46         pos.y = clampf(pos.y, 24.f, 696.f);
47     }
48     void takeDamage(float d) { hp -= d; }
49     void heal(float h) { hp = (hp + h > maxHp) ? maxHp : (hp + h); }
50     void addScore(int s) { score += s; }
51     void loseLife() { lives--; }

```

```

49 void tickShootTimer(float dt) { shootTimer -= dt; }
50 void tickIframe(float dt)      { iframeTimer -= dt; }
51 void tickShield(float dt)      { shieldTimer -= dt; }
52
53 void resetForLevel() {
54     pos = sf::Vector2f(240.f, 600.f);
55     hp = maxHp; speed = 310.f;
56     shootTimer = iframeTimer = shieldTimer = tilt = 0.f;
57     alive = true; score = 0; lives = 3;
58 }
59
60 // Operator overload: player += scoreValue
61 Player& operator+=(int scoreBonus) { score += scoreBonus; return
    *this; }
62 };

```

Listing 26: Player.h — fully encapsulated player class

A.8 main.cpp

```

1  #include "Game.h"
2  #include <iostream>
3
4  int main() {
5      try {
6          Game game;
7          game.init();
8          game.run();
9      } catch (const std::exception& ex) {
10         std::cerr << "Fatal error: " << ex.what() << std::endl;
11         return 1;
12     }
13     return 0;
14 }

```

Listing 27: main.cpp — entry point

B Build and Run Instructions

B.1 Prerequisites

- CMake 3.15 or later
- A C++17 compiler (MSVC 2019+, GCC 9+, or Clang 10+)
- SFML 2.x (fetched automatically by CMake via FetchContent)

B.2 Building with CMake

```

1  # From the project root directory
2  cmake --preset debug          # configure (creates build/debug/)
3  cmake --build build/debug     # compile
4

```

```
5 # The executable appears at build/debug/SpaceShooter
```

Listing 28: Configure and build

B.3 Running the Game

```
1 # Run from the project root so asset paths resolve correctly
2 ./build/debug/SpaceShooter
```

Listing 29: Running

B.4 Controls

Key / Input	Action
W / A / S / D or Arrow keys	Move ship
Space	Fire bullet
P or Escape	Pause / Resume
Left mouse click	Navigate menus

Table 6: Player controls